How Predictable are MLB Pitchers?

Carson Stone cstone305@gatech.edu

April 2026

1 Abstract

This projects explores the relationship between pitch type selection and situational game factors to see how predictable, if at all, certain pitchers are in Major League Baseball . In an era where pitchers are throwing with greater velocity, spin, and pitch diversity than ever before, hitters must make split-second decisions with limited information.

The goal of this study is twofold: to evaluate how accurately pitch types can be predicted based on game context (e.g., balls, strikes, outs, pitch sequence, etc.), and to identify which situational features most influence a pitcher's next selection. Using pitch-level StatCast data from the top 20 pitchers of the past 2 seasons, this project sets to build a classification model to explore the relationship between pitch selection and situational game data for the top pitchers in MLB. It also aims to build an effective predictive model that could be used in practice to provide higher quality insights to hitters.

Basic situational game factors like balls, strikes, outs and inning proved to be some of the most useful features in predicting pitch type. Lagging pitch sequence features were also significant. Several models were trained and tuned on data from individual pitchers, and Random Forest Classification using custom class weighting ended up producing the best results in terms of average accuracy (42 percent), F1 Score (40 percent), and log loss (1.35). XGboost and Logistic Regression provided similar, but slightly less impressive results. Given pitchers throw 5.2 pitches on average, these scores indicate this model could help explain pitch selection beyond each pitcher's inherent distribution.

2 Background and Motivation

One of the biggest challenges hitters in baseball face is adjusting to different pitch types in the blink of an eye. Even the best hitters in the world only get a hit in about 30 percent of their at-bats. Pitchers in today's game not only throw more pitch types than ever before [1] but are throwing them with more speed and rotation. This project aims to build a multiclass classification model for each pitcher that can allow hitters to better understand and be able to predict pitch type tendencies based on a game situation for a specific pitcher: I aim to address two preliminary research questions:

- How effectively, if at all, can the pitch type of the upcoming pitch be predicted based on a specific game situations? (i.e. number of outs, strikes, inning etc.)
- Which game scenario factors (features) are most significant in predicting the next pitch?

The first question will be the most challenging since pitchers can throw anywhere between 3-7 different pitch types. Predictive power for multiclass models becomes less effective the more classes that are introduced. We will have to be careful of which metrics are chosen to determine the efficacy of the model. Each pitcher's pitch distribution will serve as a rough baseline for model performance. For example, if the pitch most thrown by a pitcher was a fastball at 35

As a subsequent exercise, building this model would allow us to evaluate feature importance. Even if our resulting model doesn't prove to be useful from a predictive standpoint, I hope to at least be able to take away some insights around the features most

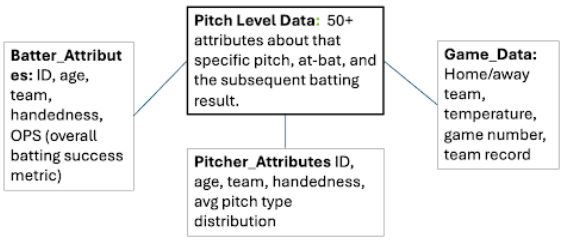

Figure 1: Data Structure

influencing pitch selection, and how much it varies pitcher to pitcher.

3 Data Sources

The pybaseball API in python [2] will be utilized to extract all the data for this report. Pybaseball provides a seamless connection to Baseball Savant, the most robust stat-keeping application for Major League Baseball. Specifically, we will be using the pitch-level data that is recorded by an advanced camera tracking system called StatCast, and loaded into Baseball Savant in real time. We will also be utilizing some player information endpoints to get player id's and other basic information about the pitchers and hitters. Below is a quick diagram of how the data from each of these endpoints will be joined together

4 Proposed Methods

4.1 Feature Selection/Reduction

- **EDA:** Significant time was spent exploring the 100+ initial features. Many were weeded out by default on the account that they take place during or after the pitch was thrown. There were over 40+ features describing either the pitch location or the hitting result of the pitch (i.e. launch angle, swing speed, batting result etc.) which would obviously not be helpful to predicting pitch type.

  During this exercise it was helpful to ensure some of the most basic game features could at least reveal some patterns for various pitch types. We plotted data for the 2025 leader in pitches thrown Logan Webb. We can observe that he appears to show different pitch distributions by inning and ball/strike count. This gives us a good indicator that there are indeed patterns that could be captured by a classification model that could help make more informed decisions.

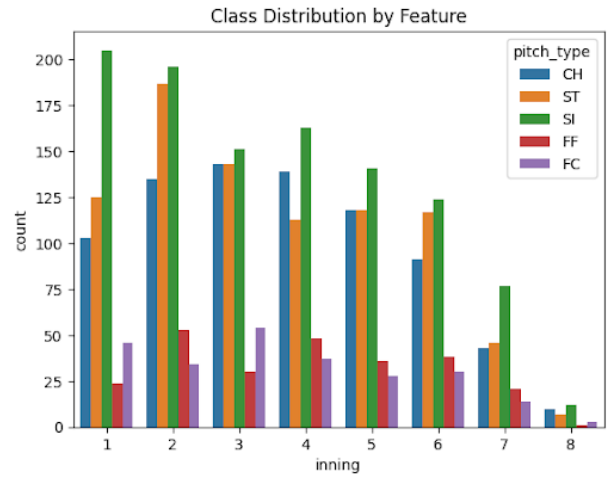

Figure 2: Enter Caption

- **Correlation**: There were many features that were highly correlated to each other. These were explored in a Pearson correlation heatmap. Most notably these were 'game length' indicators like inning, pitch number, at bat number and several others. All of them essentially are telling us how far into the game we are, so ideally only one of these would be needed. I chose inning since we had already explored its relationship with pitch selection, and things like pitch number seemed too granular.

- **Random Forest**: After the initial features were eliminated, and correlated features were identified, the features were further examined with the random forest feature significance package from scikit-learn. This method builds a random forest

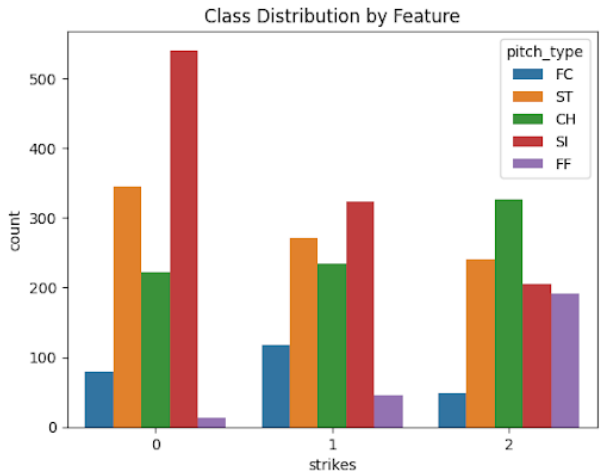

Figure 3: Strike Count v. Pitch Distribution

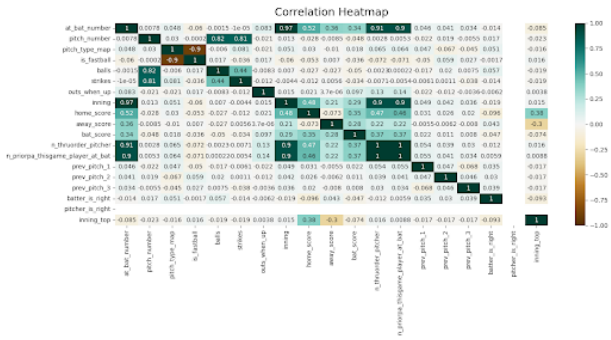

Figure 4: Feature Correlation Heat Map

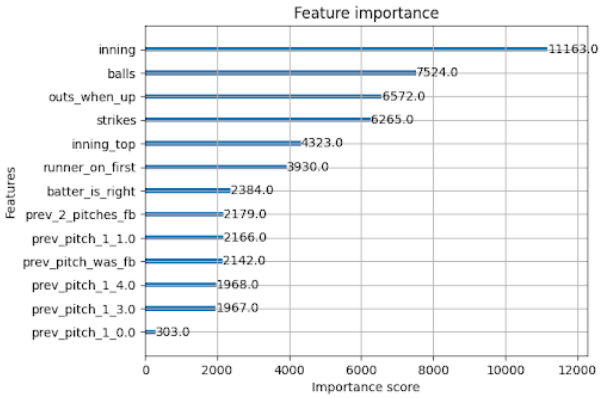

Figure 5: Feature Importance

model and computes significance scores based on how much each feature reduces node impurity on average. This was run on 10 different pitchers to see what features they had in common. This confirmed some of the more common sense features like balls, strikes, outs, and game progress were most influential. This will provide a good enough starting point for modeling and we can iteratively add/drop more features once we establish a baseline.

4.2 Model Scoring

Before testing different classification models, it was important to land on a metric, or set of metrics, to determine model efficacy. I've selected the below metrics based on the multi-class nature of the task and how they tie into the overarching goals of this project

- **Log Loss:** This will be our primary scoring metric for how "good" our model is at predicting pitch type in the first place. We will be computing a baseline log-loss using the data's natural distribution of pitch types, then comparing our model's log loss to see if we are improving over this baseline. Since accuracy alone could be misleading, log loss gives is able to give more weight to the quality the underlying prediction probabilities

- **Accuracy**: Accuracy will still be looked at as a quick gut check for model quality. However often a pitcher throws their most common pitch (i.e. 26
- **F1-Score**: This will give more context than just raw accuracy, by accounting more for the class imbalance in the data. The F1-score is the mean of precision and recall, which penalizes false positive/negatives more.
- **Confusion Matrix**: We will analyze a confusion matrix for several pitchers to get a quick visual cue of model quality and look for unwanted patterns with certain pitch types.

4.3 Model Selection

These 4 model types were used to train on the data and compare results with our metrics outlined above.

- **Logistic Regression:** This is on the simpler end of classification models in terms of complexity and explainability. Although usually preferred for binary classification tasks, it is worth testing for its ease of interpretation and well known assumptions
- **K-Nearest Neighbors:** Cross validation was used to choose the optimal K value which was k=12. KNN is another simple approach but uses euclidean distance to select classes rather than a link function. It's worth testing due to its simplicity and it doesn't make any assumptions about the data distribution
- **Random Forest**: Random forest using bootstrap sampling to build many simultaneous classification trees, then averages their predictions together. This method tends to reduce the risk of overfitting, especially for smaller datasets. Grid search cross validation will be used to determine the optimal parameters.
- **XGBoost**: This is an ensemble method that builds many decision trees in succession by correcting the errors of the previous tree. It is similar to random forest in that its robust to overfitting by considering many smaller decision trees, but differs in its architecture. It is often considered better for larger datasets where there is enough runway to build a signal

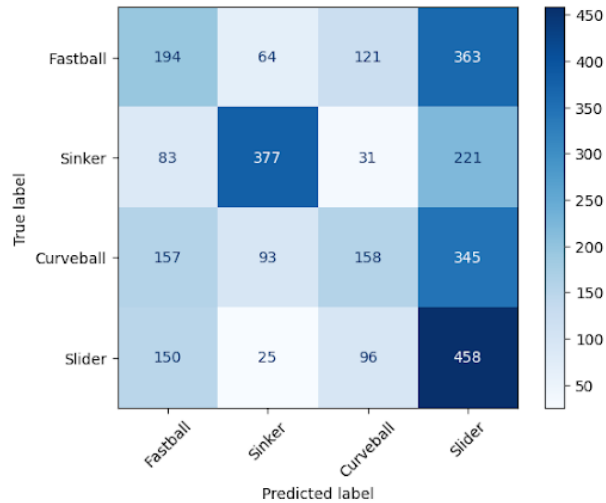

Figure 6: Confusion Matrix for Chris Sale

5 Analysis and Results

One method of feature engineering that was tested early on was accounting for class imbalance. This is often something that needs to be remedied in classification problems to avoid playing favorites and making sure smaller classes are represented. I had initially tried two methods at reducing class imbalance before realizing it did not fit our problem. At first, SMOTE technique was tested which generates artificial samples to balance out the classes represented. This was tested with an XGBoost model, and seemed like an immediate red flag when the model essentially returned noise in the confusion matrix.

Sure, the lesser thrown pitch types were well represented here, but we were losing a sense of true frequencies. If a pitcher throws 4 pitches, and a curveball only 5

Due to the nature of the data I decided to pursue a middle ground approach by using custom weights that gave the smaller classes some consideration,

	Logistic Regression			
	Accuracy	F1 Weighted	Log Loss	Log Loss v Baseline
Avg	0.4015	0.3932	1.3696	0.0366
Min	0.2342	0.2427	1.0071	-0.0730
Max	0.5415	0.5199	1.8682	0.1133

	Xgboost Regression			
	Accuracy	F1 Weighted	Log Loss	Log Loss v Baseline
Avg	0.4230	0.4020	1.4092	-0.0030
Min	0.2301	0.2405	1.0501	-0.0705
Max	0.5200	0.4917	2.0520	0.0531

	Random Forest Regression			
	Accuracy	F1 Weighted	Log Loss	Log Loss v Baseline
Avg	0.4215	0.4077	1.3516	0.0547
Min	0.2758	0.2764	1.0077	-0.0458
Max	0.5219	0.5014	1.8501	0.1313

	K Nearest Neighbors			
	Accuracy	F1 Weighted	Log Loss	Log Loss v Baseline
Avg	0.3874	0.3750	8.7650	-7.3587
Min	0.2120	0.2211	8.0224	-8.9949
Max	0.4858	0.4726	10.9763	-6.6564

Figure 7: Model Scoring Results

without ignoring the data's distribution. Removing class weights altogether often resulted in certain pitch types getting zero predictions. These custom weights take the frequency of each class and apply square root so we are able to give a boost to the smaller classes, while not telling the model that all pitches are equally likely.

For the modeling piece, the results can be observed below. Random Forest ended up producing the best all-around results, while Logistic Regression and XGBoost also performed quite well. These numbers are averaged over the top 20 pitchers. What "good" means can vary depending on the pitcher, but these different models were trained over the same data so I feel confident comparing these results in aggregate.

K Nearest Neighbors, using k = 12, was competitive in terms accuracy but the quality of the those predictions couldn't be assessed properly with the high log-loss. KNN will often assign 0.0 for probabilities of certain classes if there are no members of that class within k points. As opposed to the other models we tested where all classes will have some probability ¿ 0, even if very small. Because of this, log loss is not a good metric for KNN evaluation

Random Forest had a substantial edge in log loss (1.31 average log loss) versus the other models, while it performed similarly in terms of accuracy and weighted F1-score (42 percent and 40 percent respectively). The 20 pitchers I trained on threw anywhere between 4 and 7 different pitches, with an average of 5.25 different pitch types. The model efficacy on all metrics had a positive correlation to number of classes as expected.

The RF model was tuned using grid search cross validation and the below parameters were used:

6 Conclusions

Given the inherent challenge of predicting up to 7 classes, I feel like the model scoring ended up being more impressive than I had thought. Of course the model quality will depend entirely on each pitcher. Perhaps there are some pitchers whose tenancies would better suit a more linear pattern like a logistic regression model. There could be some pitchers that just are not predictable. We did train on one pitcher, Garrett Crochet, who was the only pitcher whose log loss across all models performed substantially worse than their baseline. Crochet also happened to have one of the most impressive seasons for any pitcher in 2025, coming in with a 2.59 ERA and 6.3 wins above replacement. This would suggest that pitchers who are less predictable, are more successful. While intuitive, this narrative doesn't quite hold up with every pitcher we trained on. Max Fried for example appeared to be one of the more predictable pitchers based on model scoring metrics, but his success speaks for itself.

In addition to predictive power, these models also showed the significance of various features of the game situation. Although a predictive model may not be practical to use in real time, viewing these relationships between counts, outs, innings, and pitch sequences is where batters could gain some real, usable insights on their pitching opponents. Logan Webb (figure 3) shows a strong tendency to throw sinker on a zero strike count, while it becomes an afterthought pitch on a 2-strike count. Most pitchers I sampled during EDA showed a some tendency whether it was based on count, game-length or better handedness.

Ultimately, pitchers are trying to be unpredictable.

They are in a constant mind game with the batter about what's coming next. It would be naive to think each pitcher's decision making could be boiled down to a machine learning model. However, classifying pitch types does provide some useful insights on pitcher tendencies based on game factors, and training on this data can provide a reasonable amount of predictive power for the upcoming pitch.

7 Credits

- Kepner, T. (2015, October 6). Pitching evolution and revolution. Our Game. https://ourgame.mlblogs.com/pitching-evolution-and-revolution-efd3a5ebaa83
- Pybaseball python library: https://github.com/jldbc/pybaseball/tree/master/docs
- My full github repository for this project: https://github.com/cstone51/MLB-Statcast-Pitch-Prediction